

Project 2 - Marking Rubric

Functionality — Learning Outcomes 1, 2 (60%)

Part 1 — Data Model (5%)

Criteria	2.5–2	1.5	1	0.5–0	Mark
Attribute and Character Classes [2.5 marks]	All classes are implemented with correct fields, methods and properties.	Minor issues in class implementation.	Significant issues in class implementation.	Major errors in class implementation.	
CharacterSeeder Class [2.5 marks]	SeedCharacters() is implemented correctly and returns at least 20 characters, each with a full and unique set of attributes.	Minor issues in the seeder implementation.	Significant issues in seeder implementation.	Major errors in seeder implementation.	

Part 2 — Game Logic (10%)

Criteria	10–8	7.5–6.5	6–5	4.5–0	Mark
Question, QuestionSeeder and GameBoard Classes	All classes are implemented with correct fields and all required methods: Matches, EliminateCharacters, GetRemainingCharacters and IsOnlyOneRemaining.	Minor issues in class implementation or methods.	Significant issues in class implementation or methods.	Major errors in class implementation or methods.	

Marking breakdown:

- Question class and Matches method [2 marks]
 - QuestionSeeder class [2 marks]
 - EliminateCharacters method [2 marks]
 - GetRemainingCharacters method [2 marks]
 - IsOnlyOneRemaining method [2 marks]
-

Part 3 — Players and Serialisation (10%)

Criteria	6–5	4.5–4	3.5–3	2.5–0	Mark
Player, HumanPlayer and ComputerPlayer Classes [6 marks]	All classes are correctly implemented with proper inheritance, fields, methods and properties. TakeTurn() is correctly overridden in both derived classes.	Minor issues in class implementation.	Significant issues in class implementation.	Major errors in class implementation.	

Criteria	4–3.5	3	2.5–2	1.5–0	Mark
GameRecord and GameHistory Classes [4 marks]	GameRecord and GameHistory are correctly implemented. Save and Load methods correctly serialise and deserialise records to and from game_history.json using System.Text.Json.	Minor issues in one or more methods.	Significant issues in serialisation implementation.	Major errors in serialisation implementation.	

Part 4 — Windows Forms UI (15%)

Criteria	15–12	11.5–10	9.5–7.5	7–0	Mark
Form1 Functionality	All specified functionalities are correctly implemented.	Some functionalities are missing or have minor issues.	Several functionalities are incomplete or have significant issues.	Most functionalities are missing or not functional.	

Marking breakdown:

- Seed characters and questions in constructor [0.5 mark]
- Display character grid on the human player's board [1 mark]
- Visual elimination by clicking [1 mark]
- Display remaining non-eliminated characters [0.5 mark]
- Question selection via ComboBox [1 mark]
- Human player asks a question and board updates [1.5 marks]
- Computer player's turn with Yes/No prompt and board update [2 marks]
- Human player final guess with win/loss message [1.5 marks]
- Computer player final guess with win/loss message [1 mark]
- Save GameRecord to game_history.json [1 mark]
- Display past game records in DataGridView [1 mark]
- New Game button [0.5 mark]
- Quit button [0.5 mark]

Part 5 — Additional Features (20%)

Criteria	20–16	15.5–13	12.5–10	9.5–0	Mark
Additional Features	Four additional features are correctly implemented.	Some features are missing or have minor issues.	Several features are incomplete or have significant issues.	Most features are missing or not functional.	

Marking breakdown:

- Medium features: 3 marks each
- Hard features: 5 marks each

Only 4 will be marked.

Comments — Functionality

Code Quality and Best Practices — Learning Outcome 2 (40%)

Criteria	5–4	3.5	3–2.5	2–0	Mark
Naming Conventions	File, class, method, variable, constant and property names are clear, descriptive and consistent across the codebase.	Minor inconsistencies in naming conventions.	Significant inconsistencies in naming conventions.	Naming conventions are not followed.	

Criteria	8–6.5	6–5	4	3.5–0	Mark
Comments	Code is well-commented using XML documentation format that explains complex logic and clarifies the purpose of classes, methods or complex sections.	Minor issues in XML documentation comments.	Significant issues in XML documentation comments.	Missing or incorrect XML documentation comments.	

Criteria	5–4	3.5	3–2.5	2–0	Mark
Code Formatting	Consistent indentation and spacing used across the codebase with proper indentation for code blocks, standard format for braces and appropriate blank lines between sections.	Minor inconsistencies in code formatting.	Significant inconsistencies in code formatting.	Poor or non-consistent code formatting.	

Criteria	5–4	3.5	3–2.5	2–0	Mark
----------	-----	-----	-------	-----	------

Criteria	5–4	3.5	3–2.5	2–0	Mark
Dead Code	No dead or unused code is present in the application.	Minor instances of dead or unused code.	Significant instances of dead or unused code.	Widespread presence of dead or unused code.	

Criteria	15–12	11.5–10	9.5–7.5	7–0	Mark
Performance and Scalability	Code is optimised for performance using OOP principles, efficient algorithms and data structures, and error handling to minimise bottlenecks.	Minor inefficiencies in the performance and scalability approach.	Significant inefficiencies in the performance and scalability approach.	Poor or non-efficient performance and scalability approach.	

Error handling examples:

- When adding a custom character, the name should not be empty and should not already exist in the character pool
- Question selection should be validated — the player must select a question before submitting their turn
- Game history file errors, i.e., a missing or corrupted game_history.json, should be caught and handled gracefully, starting with an empty history if needed
- Final guess validation — the player must select a character name before submitting a guess

Comments — Code Quality and Best Practices

Criteria	2–1.5	1.5	1	0.5–0	Mark
Class Diagram	A clear and accurate class diagram of the application generated using Visual Studio's built-in tools, showing all classes, methods and properties.	Minor issues or inaccuracies in the class diagram.	Significant issues or inaccuracies in the class diagram.	Class diagram is missing or unclear.	

Comments — Class Diagram